Module 8 — Conversation History & Memory

25 MCQs with Answers, Explanations & Learning Conclusions

MCQ 1 — Stateless LLM Calls

Question: What happens when we call llm.invoke() two times independently, without supplying previous conversation history?

A. The LLM automatically remembers the first conversation.

B. The LLM saves the conversation in a database.

C. The second call does not automatically receive the first conversation.

D. The LLM automatically trains itself using the first conversation.

Correct Answer: C

Explanation: Each independent llm.invoke() call receives only the information supplied in that call. Previous conversation history is not automatically included.

Learning Conclusion: Independent LLM calls are stateless by default. The application must maintain and supply previous conversation history.

MCQ 2 — Same LLM Object

Question: Consider the following Python code:

```
llm = ChatOpenAI(model="gpt-4o-mini")


llm.invoke("My name is Durga.")


response = llm.invoke("What is my name?")
```

Why does the second call not automatically remember the name, even though we are using the same llm object?

A. The llm object is recreated automatically after every call.

B. The same llm object does not automatically maintain conversation history.

C. ChatOpenAI does not support conversation history.

D. We must change the model name for every call.

Correct Answer: B

Explanation: The llm object is used to communicate with the model. Simply reusing the same object does not mean that previous messages are automatically stored or passed to the next call.

Learning Conclusion: Same LLM Object ≠ Same Conversation History.

MCQ 3 — Manual Conversation History

Question: In a manually managed LangChain chatbot, which is the correct order of operations for maintaining conversation history?

A. Invoke model → Add HumanMessage → Add AIMessage

B. Add AIMessage → Invoke model → Add HumanMessage

C. Add HumanMessage → Invoke model with history → Add AIMessage

D. Invoke model → Clear history → Add HumanMessage

Correct Answer: C

Explanation: First, add the user's message to history. Next, send the history to the model. Finally, save the generated AI response for future conversations.

Learning Conclusion: ADD HUMAN MESSAGE → INVOKE MODEL → SAVE AI RESPONSE.

MCQ 4 — Saving AI Responses

Question: Consider the following LangChain code:

```
messages = []


messages.append(
    HumanMessage(content="My name is Durga.")
```

)

response = llm.invoke(messages)

print(response.content)

What is the main problem if we forget to execute messages.append(response)?

A. The LLM will stop generating responses.

B. The AI response will not be stored in conversation history.

C. The HumanMessage will be automatically deleted.

D. The program will always raise a Python syntax error.

Correct Answer: B

Explanation: The LLM generates a response, but without messages.append(response), that response will not become part of the stored history.

Learning Conclusion: A complete conversation history should contain both HumanMessage and AIMessage objects.

MCQ 5 — Conversation Window

Question: Consider the following Python code:

```
history = [
    "M1", "M2", "M3",
    "M4", "M5", "M6",
    "M7", "M8"
]

recent_history = history[-4:]

print(recent_history)
```

What will be the output?

A. ['M1', 'M2', 'M3', 'M4']

B. ['M3', 'M4', 'M5', 'M6']

C. ['M5', 'M6', 'M7', 'M8']

D. ['M4', 'M5', 'M6', 'M7']

Correct Answer: C

Explanation: Negative slicing with [-4:] selects the last four elements from the list. Therefore, the result contains M5, M6, M7 and M8.

Learning Conclusion: Negative slicing is useful for selecting recent messages from conversation history.

MCQ 6 — Conversation History

Question: In a window-based chatbot, consider the following code:

```
messages = []  # Complete history


MAX_MESSAGES = 6


recent_messages = messages[-MAX_MESSAGES:]


response = llm.invoke(recent_messages)
```

Suppose messages already contains 20 messages. Which statement is correct?

A. The application permanently deletes the first 14 messages.

B. The application stores 20 messages but sends only the last 6 to the model.

C. The model automatically receives all 20 messages.

D. Python raises an error because the history contains more than 6 messages.

Correct Answer: B

Explanation: Assuming the history already contains 20 messages when slicing happens, messages[-6:] selects only the latest six messages. The complete list still contains all 20 messages.

Learning Conclusion: Complete Stored History ≠ History Sent to Model.

MCQ 7 — Messages vs Turns

Question: In a LangChain chatbot, suppose one complete conversation turn contains one HumanMessage and one AIMessage.

If we want to keep the last 3 complete conversation turns, how many messages should we normally select?

A. 3 messages

B. 4 messages

C. 6 messages

D. 9 messages

Correct Answer: C

Explanation: One complete conversation turn normally contains two messages: one human message and one AI response. Therefore, three turns contain six messages.

Learning Conclusion: 1 Complete Turn = 2 Messages, assuming a normal Human–AI exchange.

MCQ 8 — Legacy Memory APIs

Question: What is the main purpose of ConversationBufferMemory in older LangChain applications?

A. Keep only the most recent two messages.

B. Store the complete conversation history in a buffer.

C. Permanently train the LLM using previous conversations.

D. Convert conversation messages into embeddings.

Correct Answer: B

Explanation: ConversationBufferMemory stores the complete conversation history. As the conversation grows, passing the full history to the LLM can increase token usage and cost.

Learning Conclusion: ConversationBufferMemory = Full Conversation History. It is a legacy LangChain API.

MCQ 9 — Legacy Window Memory

Question: Consider the following legacy LangChain code:

```
memory = ConversationBufferWindowMemory(
    k=2
)
```

What does k=2 represent?

A. Keep only the last 2 individual messages.

B. Keep the last 2 conversation turns in the active history window.

C. Allow only 2 users to access the chatbot.

D. Store conversation history for 2 days.

Correct Answer: B

Explanation: In legacy ConversationBufferWindowMemory, k=2 represents the last two conversation turns, normally containing two human messages and two AI responses.

Learning Conclusion: The k parameter controls the number of recent conversation turns in legacy window memory.

MCQ 10 — Legacy Memory Methods

Question: Consider the following LangChain code:

```
memory = ConversationBufferMemory()


memory.save_context(
   {"input": "My name is Durga."},
   {"output": "Hello Durga!"}
)


result = memory.load_memory_variables({})
```

What is the purpose of load_memory_variables({})?

A. Delete the complete conversation history.

B. Save a new HumanMessage into memory.

C. Retrieve the stored conversation history from memory.

D. Permanently save the conversation into an SQLite database.

Correct Answer: C

Explanation: save_context() stores the interaction in the legacy memory object. load_memory_variables() retrieves the saved memory information.

Learning Conclusion: save_context() = SAVE; load_memory_variables() = LOAD.

MCQ 11 — Legacy Memory Coding

Question: Consider the following LangChain code:

```
memory = ConversationBufferMemory(
   return_messages=True
)

memory.save_context(
   {"input": "My name is Durga."},
   {"output": "Hello Durga!"}
```

)

result = memory.load_memory_variables({})

What is the main purpose of return_messages=True?

A. Return the conversation history as a single formatted string.

B. Return only the AI responses and ignore user messages.

C. Return conversation history as LangChain message objects such as HumanMessage and AIMessage.

D. Automatically save the conversation in a database.

Correct Answer: C

Explanation: return_messages=True makes the history available as actual LangChain message objects instead of a single formatted string.

Learning Conclusion: return_messages=True → History as LangChain Message Objects.

MCQ 12 — Legacy ConversationChain

Question: Consider the following LangChain code:

memory = ConversationBufferMemory()

conversation = ConversationChain(

```
    llm=llm,
    memory=memory
)
```

What was the main purpose of ConversationChain in older LangChain applications?

A. Automatically train the LLM using every user message.

B. Combine an LLM with conversational memory to manage chat history automatically.

C. Permanently store all conversations in an SQLite database.

D. Execute multiple LLM models in parallel.

Correct Answer: B

Explanation: ConversationChain combined an LLM with a memory component to load previous history and save new conversations automatically. It is a legacy/deprecated API.

Learning Conclusion: Legacy ConversationChain = LLM + Memory.

MCQ 13 — RunnableWithMessageHistory

Question: What is the main purpose of RunnableWithMessageHistory in LangChain?

A. Wrap another Runnable and automatically read and update conversation history.

B. Train the LLM using previous conversation messages.

C. Convert every conversation into a vector embedding.

D. Automatically create a permanent SQLite database for conversation history.

Correct Answer: A

Explanation: RunnableWithMessageHistory wraps another Runnable, retrieves previous conversation messages before execution and updates the history after execution.

Learning Conclusion: Runnable + Message History = History-Aware Runnable.

MCQ 14 — Session ID Coding

Question: Consider the following LangChain code:

```
response = chatbot.invoke(
   [HumanMessage(content="What is my name?")],
   config={
      "configurable": {
         "session_id": "durga"
      }
   }
)
```

What is the main purpose of session_id="durga"?

A. Set the name of the OpenAI model.

B. Permanently store the conversation in the database.

C. Identify and retrieve the conversation history associated with the session "durga".

D. Limit the conversation to only one message.

Correct Answer: C

Explanation: The session ID identifies the correct conversation history. The same session ID allows later calls to access the history associated with that session.

Learning Conclusion: Same Session ID → Same History; Different Session ID → Separate History.

MCQ 15 — LCEL & Message History

Question: Consider the following LangChain code:

```
chain_with_history = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="question",
    history_messages_key="history"
)
```

What is the main purpose of input_messages_key="question"?

A. It specifies the field that contains the previous conversation history.

B. It specifies the field that contains the current user's message.

C. It specifies the session ID used to identify the user.

D. It specifies the name of the LLM model.

Correct Answer: B

Explanation: input_messages_key="question" tells LangChain that the question field in the input dictionary contains the current user message.

Learning Conclusion: input_messages_key identifies the CURRENT USER INPUT.

MCQ 16 — MessagesPlaceholder

Question: Consider the following LangChain code:

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    MessagesPlaceholder("history"),
    ("human", "{question}")
])
```

```
chain = prompt | llm


chatbot = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="question",
    history_messages_key="history"
)
```

What is the purpose of history_messages_key="history"?

A. It specifies the name of the database table used to store messages.

B. It identifies the current user's question.

C. It specifies which session ID should be used.

D. It tells LangChain to supply previous messages to MessagesPlaceholder("history").

Correct Answer: D

Explanation: history_messages_key="history" identifies the prompt variable that receives previous conversation messages. It must match MessagesPlaceholder("history").

Learning Conclusion: history_messages_key connects the previous conversation history with MessagesPlaceholder.

MCQ 17 — InMemoryChatMessageHistory

Question: Consider the following LangChain code:

```
history = InMemoryChatMessageHistory()


history.add_user_message("My name is Durga.")


history.add_ai_message("Hello Durga!")
```

Suppose we stop the Python program completely and restart it. What happens to this conversation history?

A. The history is automatically restored from OpenAI servers.

B. The history is automatically saved permanently in SQLite.

C. The in-memory history is lost unless it was saved to persistent storage.

D. LangChain automatically converts the history into a JSON file.

Correct Answer: C

Explanation: InMemoryChatMessageHistory stores messages in application memory (RAM). When the application stops, the in-memory history is lost unless we save it separately.

Learning Conclusion: In-Memory History ≠ Persistent History.

MCQ 18 — Serialization

Question: Consider the following LangChain code:

```
from langchain_core.messages import (
    HumanMessage,
    message_to_dict
)

message = HumanMessage(content="Hello Durga!")

data = message_to_dict(message)
```

What does message_to_dict(message) return?

A. A JSON file saved automatically on the computer.

B. A Python dictionary representing the LangChain message.

C. A new AIMessage object.

D. An SQLite database record created automatically.

Correct Answer: B

Explanation: message_to_dict() converts a LangChain message object into a Python dictionary, which can later be converted into JSON for storage.

Learning Conclusion: Serialization = Object → Storable Format.

MCQ 19 — JSON Serialization

Question: Consider the following Python code:

```
import json


data = {
   "name": "Durga",
   "course": "LangChain"
}


json_string = json.dumps(data, indent=2)
```

What is the main purpose of json.dumps()?

A. Convert a JSON string into a Python dictionary.

B. Automatically save the Python dictionary into a JSON file.

C. Convert a Python dictionary into a JSON-formatted string.

D. Convert a Python dictionary directly into a LangChain HumanMessage.

Correct Answer: C

Explanation: json.dumps() converts a Python object into a JSON-formatted string. The indent=2 parameter improves readability by adding indentation.

Learning Conclusion: json.dumps() = Python Object → JSON String.

MCQ 20 — Deserialization

Question: Consider the following LangChain code:

```
import json

from langchain_core.messages import (
   messages_from_dict
)

python_data = json.loads(json_string)

restored_messages = messages_from_dict(
   python_data
)
```

What is the main purpose of messages_from_dict()?

A. Convert LangChain message objects into Python dictionaries.

B. Convert a JSON string directly into an SQLite table.

C. Convert Python dictionaries into a JSON string.

D. Convert a sequence of Python dictionaries back into LangChain message objects.

Correct Answer: D

Explanation: messages_from_dict() converts message dictionaries back into LangChain objects such as HumanMessage and AIMessage.

Learning Conclusion: Deserialization = Stored Format → Original Objects.

MCQ 21 — Persistent Conversation History

Question: Suppose a chatbot stores its conversation history only in a Python list. We want to preserve the conversation even after closing and restarting the application.

What should we do?

A. Use the same ChatOpenAI object every time.

B. Increase the context window of the LLM.

C. Save the conversation history in persistent storage, such as a JSON file or SQLite database, and reload it when needed.

D. Use HumanMessage instead of AIMessage.

Correct Answer: C

Explanation: Persistent storage saves the conversation outside temporary application memory. When the application restarts, previous messages can be loaded again.

Learning Conclusion: Persistent History = SAVE → RESTART → RELOAD.

MCQ 22 — Restoring JSON History

Question: Consider the following Python code:

```
import json

from langchain_core.messages import (
    messages_from_dict
)

with open("chat_history.json", "r") as file:
    data = json.load(file)

messages = messages_from_dict(data)
```

What is the correct explanation of these two operations?

A. json.load() creates a JSON file, and messages_from_dict() saves messages into it.

B. json.load() reads JSON data into Python objects, and messages_from_dict() restores LangChain message objects.

C. json.load() converts messages into embeddings, and messages_from_dict() creates an LLM.

D. Both functions automatically store the conversation in SQLite.

Correct Answer: B

Explanation: json.load() reads the JSON file and creates Python data. messages_from_dict() converts the loaded message dictionaries into LangChain message objects.

Learning Conclusion: JSON File → Python Dictionaries → LangChain Messages.

MCQ 23 — Multiple Sessions

Question: Consider the following LangChain code:

```
store = {}


def get_session_history(session_id):
    if session_id not in store:
        store[session_id] = (
            InMemoryChatMessageHistory()
        )
    return store[session_id]
```

h1 = get_session_history("durga")

h1.add_user_message("I teach Python.")

h2 = get_session_history("ravi")

h3 = get_session_history("durga")

Which statement is correct?

A. h1, h2, and h3 all refer to the same conversation history.

B. h1 and h3 refer to the same history, but h2 refers to a separate history.

C. Every call to get_session_history() creates a new history object.

D. Calling get_session_history("ravi") automatically deletes Durga's history.

Correct Answer: B

Explanation: The dictionary uses session IDs as keys. Both h1 and h3 use "durga", so they reference the same history object. h2 uses "ravi" and receives a separate history.

Learning Conclusion: Same Session ID → Same History Object; Different Session ID → Separate History Object.

MCQ 24 — Session Persistence

Question: Consider the following LangChain code:

```
store = {}


def get_session_history(session_id):
    if session_id not in store:
        store[session_id] = (
            InMemoryChatMessageHistory()
        )
    return store[session_id]
```

Suppose Durga and Ravi have separate conversation histories. We completely stop the Python application and restart it.

What happens when we call get_session_history("durga") again?

A. Durga's previous history is automatically restored because the session ID is the same.

B. LangChain automatically retrieves Durga's conversation from OpenAI servers.

C. A new empty history is created because the dictionary and in-memory objects were lost when the program stopped.

D. Durga's previous history is automatically loaded from SQLite.

Correct Answer: C

Explanation: After restarting the Python application, store = {} creates a new empty dictionary. Since there is no persistent storage, Durga's earlier messages cannot be recovered automatically.

Learning Conclusion: Same Session ID ≠ Permanent History. Persistent Storage is required.

MCQ 25 — Complete Multi-User Chatbot Architecture

Question: Suppose we want to build a LangChain chatbot with the following requirements:

- Durga and Ravi must have separate conversation histories.
- The chatbot should remember previous conversations.
- History must remain available even after restarting the Python application.
- We want to use RunnableWithMessageHistory.

Which implementation is most suitable?

A. Use a single Python list for all users and pass it directly to the LLM.

B. Use InMemoryChatMessageHistory with separate session IDs, without any persistent storage.

C. Use the same ChatOpenAI object for all users without maintaining conversation histories.

D. Use RunnableWithMessageHistory, separate session IDs, and a file-based or database-backed history implementation.

Correct Answer: D

Explanation: RunnableWithMessageHistory manages conversation history, session IDs separate conversations, and persistent storage such as JSON files or SQLite preserves messages after application restarts.

Learning Conclusion: Complete Conversation History Management = RunnableWithMessageHistory + Session IDs + Persistent Storage.

FINAL REVISION — MOST IMPORTANT CONCLUSIONS

1. Stateless LLM Calls

Independent model calls do not automatically share previous conversation history.

2. Manual Conversation History

Add HumanMessage → Invoke Model → Save AIMessage.

3. Window-Based History

Store complete history but send only the required recent messages to the LLM.

4. Legacy Memory

ConversationBufferMemory and ConversationChain are older LangChain approaches.

5. RunnableWithMessageHistory

Wraps a Runnable and manages conversation history before and after execution.

6. Session IDs

Same Session ID → Same History.

Different Session ID → Separate History.

7. In-Memory History

InMemoryChatMessageHistory keeps messages in RAM and does not automatically provide permanent storage.

8. Serialization

LangChain Message → message_to_dict() → Python Dictionary → json.dumps() → JSON String.

9. Deserialization

JSON String → json.loads() → Python Dictionaries → messages_from_dict() → LangChain Messages.

10. Persistent History

Store messages in JSON files or databases such as SQLite so that earlier conversations can be restored after application restarts.

FINAL LEARNING CONCLUSION

Friends, remember these three important responsibilities:

APPLICATION → Manages conversation history.

SESSION ID → Identifies the correct conversation.

PERSISTENT STORAGE → Preserves the conversation for future use.

Once these three concepts are clear, building a multi-user LangChain chatbot becomes much easier.

Add a complete practical revision programAdd module objectives and prerequisites